

Lecture 7: Notifications and Workflow Integration

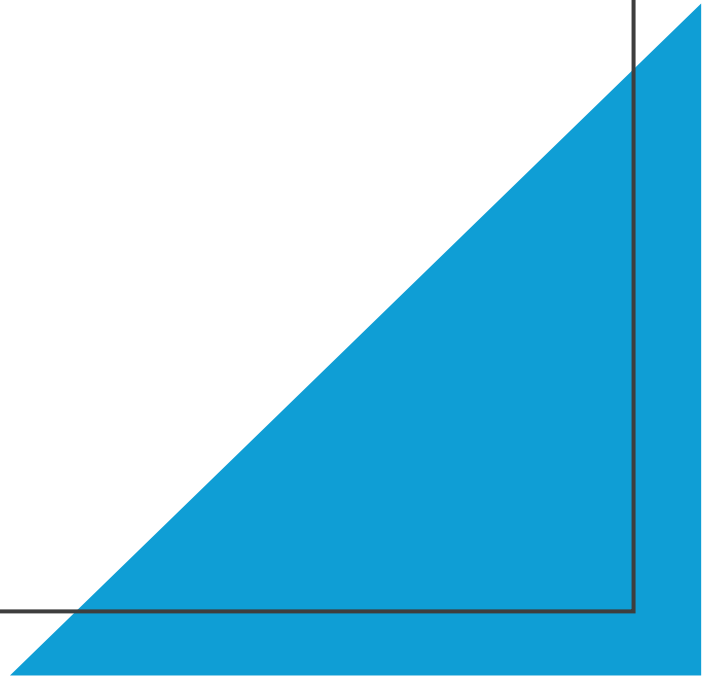
CSC 1202: Software Development Project

Dr. Peter Khisa Wakholi



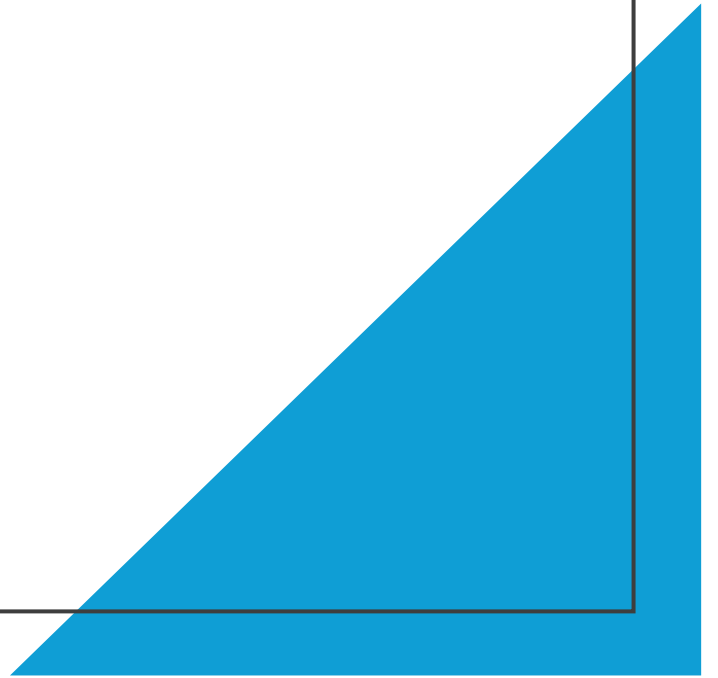
Lecture Objectives

- Understand notification systems in web applications
- Implement email and SMS notifications in Django
- Integrate notifications with workflow updates
- Implement frontend notifications using React



Importance of Notifications

- Enhances user engagement
- Provides timely updates and alerts
- Improves transparency in issue resolution



Types of Notifications



Email Notifications



SMS Notifications



In-App/Visual Notifications

Configuring Django for Email Notifications

- Setup SMTP backend (e.g., Gmail SMTP server)
- Example settings in settings.py

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'  
EMAIL_HOST = 'smtp.gmail.com'  
EMAIL_PORT = 587  
EMAIL_USE_TLS = True  
EMAIL_HOST_USER = 'your_email@gmail.com'  
EMAIL_HOST_PASSWORD = 'your_password'
```

Creating Email Notifications

Django built-in function `send_mail()`

```
from django.core.mail import send_mail

def send_notification_email(subject, message, recipient_list):
    send_mail(subject, message, EMAIL_HOST_USER, recipient_list)
```

Triggering Email Notifications

Integrate within issue update workflow:

```
subject = 'Issue Status Updated'
message = f"Issue '{issue.title}' status is now '{issue.status}'."
recipient = [issue.reporter.email]
send_notification_email(subject, message, recipient)
```



Introduction to SMS Notifications

Use third-party
services (e.g., Twilio)

Reliable and instant
user communication

Twilio Setup for SMS Notifications

1

Create Twilio Account

2

Obtain Account SID, Auth Token, Twilio Phone Number

3

Install Twilio Python SDK: `pip install twilio`

SMS Notifications Implementation

Example function to send SMS via Twilio:

```
from twilio.rest import Client

def send_sms_notification(to, body):
    client = Client('TWILIO_SID', 'TWILIO_AUTH_TOKEN')
    message = client.messages.create(
        body=body,
        from_='your_twilio_number',
        to=to
    )
```

Workflow Integration using Django Signals

Automate
notifications
when issues are
updated

Introduction to
Django Signals
(post_save)

Django Signals – Practical Example

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Issue

@receiver(post_save, sender=Issue)
def notify_on_issue_update(sender, instance, created, **kwargs):
    if not created:
        subject = 'Issue Status Updated'
        message = f"Issue '{instance.title}' status updated to '{instance.status}'."
        send_notification_email(subject, message, [instance.reporter.email])
        send_sms_notification(instance.reporter.phone_number, message)
```

Benefits of Workflow Integration

Automatic
updates reduce
manual
interventions

Immediate user
notifications
enhance user
satisfaction

Streamlined and
transparent issue
resolution

Frontend Notifications

- Visual feedback for immediate user interaction
- Using React libraries like React Toastify

```
function notify(message) {  
    toast.success(message);  
}
```

```
// Usage example
```

```
notify('Issue status updated successfully!');
```

Assignment and Programming Task Overview

Assignment:

- Develop and rigorously test the notification systems within the AITS project.
- Ensure comprehensive coverage: verify all user roles (students, lecturers, academic registrar) receive notifications at critical workflow events (issue logging, assignment, resolution, and status updates).

Programming Task:

- Implement Django-based email notifications triggered by issue status updates, ensuring notifications are sent automatically.
- Integrate React Toastify for visual notifications to enhance user experience, providing immediate feedback within the frontend upon issue submission, updates, or system actions.